

PDF Studio

User Manual

Version 3.2-alpha12

Internal development alpha - release assurance in progress

Leon Priest · github.com/7h3v01d

Application source: Apache-2.0; binary distribution not approved

Contents

1	About PDF Studio
2	Installing and running
3	The interface
4	Appearance and accessibility
5	Opening documents
6	Viewing and navigating
7	Searching
8	Annotations and markup
9	Signatures and stamps
10	Filling in existing PDF forms
11	Editing text in scanned pages
12	Redactions
13	Managing pages
14	Merging, splitting, extracting
15	Password protection
16	OCR - making scans searchable
17	Exporting pages and document content
18	Saving and printing
19	File associations (Windows)
20	Keyboard shortcuts
21	Building from source
22	Troubleshooting
23	Licence and credits

1. About PDF Studio

PDF Studio is a free, full-featured PDF reader and editor for Windows, built with Python, PyQt6, and PyMuPDF.

PDF Studio does not contain a trial, activation, licence key, or feature paywall. The application source is released under Apache-2.0. Distribution of a bundled executable remains under review because third-party runtime components carry their own licence obligations.

Internal alpha: version 3.2-alpha12 includes five data-integrity remediation passes plus the first release-assurance layer. Save recovery now verifies deletion of original transaction copies and warns prominently whenever sensitive pre-redaction backups remain. Recovery data and Office-import caches live only in marker-owned application directories, but packaged-GUI, dependency, licensing, and clean-machine release gates remain. See `KNOWN_ISSUES.md` .

Highlights

- View, annotate, mark up, and sign PDFs
- Fill in interactive PDF forms
- Insert, delete, reorder, rotate, merge, split, and extract pages
- Replace selected text in scans with reversible or permanent edits
- Apply true redactions
- AES-256 password protection
- OCR scanned documents to make them searchable
- Open Word and Excel documents
- Export PDF pages to images, or PDF content to Word and Excel
- Two accessibility-focused themes with an app-wide text-size control

2. Installing and running

Requirements

- Windows 10 or 11 (the app also runs on Linux/macOS from source)
- Python 3.11 (*only if running from source; required by the validated build workflow*)

Option A - Run the built executable

Double-click `PDF Studio.exe` . Nothing to install. A branded startup image appears immediately, remains visible for at least 4.5 seconds while the main window is prepared, then fades away. The splash is cosmetic: if its image cannot be loaded, PDF Studio continues with a normal launch.

Option B - Run from source

```
cd PDF_Studio/src
python -m venv .venv
.venv\Scripts\activate          # Windows
pip install -r ../requirements.txt
python pdf_reader.py
```

Optional features

Some features rely on additional components. The app works fine without them and will tell you what is missing if you try to use one.

Feature	Python packages	External programs
Open Word / Excel	<i>(none)</i>	Microsoft Office (best fidelity) or LibreOffice
Export pages to images	<i>(included)</i>	-
Export to Word	pdf2docx	-
Export to Excel	tabula-py , openpyxl , pandas	Java
OCR	pytesseract , Pillow	Tesseract-OCR (auto-detected; no PATH edit)

3. The interface

Area	What it does
Menu bar	File, Edit, View, Pages, Tools, Help
Main toolbar	Open, Save, Print, page navigation, zoom, rotate, full screen, search
Markup toolbar	Note, Highlight, Underline, Strikethrough, Draw, Eraser, colour, Signature, Stamp
Navigation panel (left)	Contents, Bookmarks, Annotations, Thumbnails
Page area (centre)	The document itself
Status bar (bottom)	Current page, zoom, rotation, view mode

The **Navigation panel** has four collapsible sections. Click a section header to expand or collapse it; drag the dividers to resize. Its layout is remembered between sessions. Toggle the whole panel with

F4.

An asterisk (*) in the title bar means you have **unsaved changes**.

4. Appearance and accessibility

PDF Studio was built with low-vision readers in mind. All settings persist between sessions.

Themes

View > Appearance, then choose:

- **High-Contrast Light** - near-black text on white (*default*)
- **Dark Industrial** - light text on an obsidian background

Neither is universally "better" for low vision - it varies by individual and by condition. Try both.

Text size

View > Appearance > Text Size: Medium, Large (*default*), or **Extra Large**. This scales menus, toolbars, panels, dialogs, and form-field text across the whole application, and enlarges toolbar icons to match.

Typeface

All interface text is set in **Atkinson Hyperlegible**, a typeface designed by the Braille Institute to maximise character distinction for low-vision readers. It is bundled with the app - no installation needed.

Dark page background

View > Dark Background inverts the *document page* itself (separate from the app theme), which can reduce glare on dense white pages. You can combine a light interface with a dark page, or vice versa.

5. Opening documents

File > Open... (`Ctrl+O`), or click **Open** on the toolbar. The **Open** button's dropdown arrow lists your 10 most recent files.

Supported formats

Type	Extensions
PDF	.pdf
Word	.docx , .doc , .rtf , .odt
Excel	.xlsx , .xls , .ods , .csv

How Word and Excel files are opened

Office documents are **converted to PDF** for viewing and markup. The title bar shows the original filename with (*imported*) appended.

PDF Studio picks the highest-fidelity converter available:

- 1. Microsoft Word / Excel** via COM automation - this is Office performing its own *Save as PDF*, so the result is an exact reproduction. Used automatically when Office is installed. Requires `pywin32` .
- 2. LibreOffice** (headless) - free, very faithful, but not guaranteed pixel-identical to Word for complex layouts.
- 3.** If neither is installed, the app explains what to install rather than producing a poor conversion.

Note: Imported documents are opened as *PDFs*. The converted PDF is a temporary viewing cache, not a user-owned destination. `Ctrl+S` therefore opens **Save As** and suggests `<original-name>.pdf` beside the Office source. The cache is removed when the imported session ends. This is a view-and-markup path, not round-trip Word editing.

Password-protected PDFs

If a PDF is encrypted, you'll be prompted for the password on open.

6. Viewing and navigating

Action	How
Next / previous page	Next / Prev , or > / <
Jump to page	Type the number in the toolbar page box, press Enter
First / last page	Ctrl+Home / Ctrl+End
Zoom in / out	Zoom + / Zoom - , Ctrl++ / Ctrl+- , or Ctrl+Wheel
Fit width / page	Fit W / Fit Pg (Ctrl+Shift+H / Ctrl+Shift+F)
Set an exact zoom	Type a percentage in the zoom box
Rotate 90°	Rotate (Ctrl+R)
Full screen	Full (F11)
Continuous scroll	View > Continuous Scroll
Show/hide nav panel	F4

Thumbnails in the navigation panel jump to any page on click, and support drag-and-drop reordering (see §12).

Bookmarks: **Ctrl+B** or **Pages > Add Bookmark** bookmarks the current page. Use **+ Add** / **- Remove** in the Bookmarks section.

Contents shows the PDF's own table of contents (outline), when it has one.

7. Searching

1. Click the **Search** box (**Ctrl+F**).
2. Type your text and press **Enter** .
3. Move between hits with **Next** / **Prev**, or **F3** / **Shift+F3** .

Matches are highlighted on the page. Search covers the whole document.

If a scanned document returns no results, it has no text layer - run **OCR** first (§15).

8. Annotations and markup

Select a tool from the markup toolbar, then use the mouse on the page. Press **Esc** to put the tool down.

Tool	Use
Note Note	Click the page to place a sticky note; type its text
Highlight	Drag across text
Underline	Drag across text
Strikethrough	Drag across text
Edit Draw	Freehand ink
Eraser	Removes nearby markup
(selected) (colour)	Sets the markup colour (remembered between sessions)

The **Annotations** section of the navigation panel lists every annotation in the document. Click one to jump to it, or delete it from there.

Before save, new notes and markup remain deferred and undoable. After a successful save they become native PDF annotations and the legacy sidecar entry is retired, so the same item is not drawn or listed twice after reopening.

Markup, native annotation deletion, signatures, and stamps support **Undo/Redo** (**Ctrl+Z** / **Ctrl+Y**). Native-image undo is memory-bounded to prevent unlimited RAM growth on large documents.

9. Signatures and stamps

These are **visual ink signatures** - an image stamped onto the page. They are *not* cryptographic/digital signatures (PDF certificate signing), and PDF Studio does not claim to provide those.

Adding a signature

Click **Signature Signature** on the markup toolbar. You have two modes:

Import image file *(recommended)*

1. Choose **Import image file**.
2. Click **Choose image...** and select a PNG/JPG of your signature.
3. Leave **Remove white background** ticked for scans or phone photos - it keys

out the white paper so only the ink is placed.

4. Click **Add Signature**, then **click the page** where it should go.

Draw signature

1. Draw in the canvas with the mouse. Adjust **Thickness** and **Ink Colour**;

Clear to start over.

2. Click **Add Signature**, then **click the page** where it should go.

Drag and drop

The fastest route: **drag an image file from Explorer straight onto the page**. It is placed at the drop point. If the image has no transparency, the white background is removed automatically.

Sizing

Signatures are placed at a sensible default width (~200pt), aspect-ratio preserved, capped at half the page width. Use **Ctrl+Z** to undo a misplacement.

Stamps

Redaction Stamp inserts a text stamp (e.g. *APPROVED*, *DRAFT*) - you'll be prompted for the text, then click to place it.

Signatures and stamps are embedded permanently on **Save**.

10. Filling in existing PDF forms

PDF Studio automatically detects interactive **AcroForm** fields already built into a PDF. It places live controls over the page and lists every field in the **Forms** section of the left navigation panel. Double-click a listed field to jump to it.

Field type	Behaviour
Text (single-line)	Click and type
Text (multi-line)	Click and type multiple lines
Checkbox	Click to tick or untick
Radio button	Click to select within its group
Dropdown (combo)	Open and choose an option
List box	Select one visible option
Signature field	Detected and identified; cryptographic signing is not yet supported

Field type	Behaviour
PDF push button	Shown but disabled; embedded actions and JavaScript are not executed

Fillable fields are outlined in blue by default. Read-only fields have a distinct appearance and cannot be changed. Turn **Highlight fillable fields** off in the Forms panel when you want an unobstructed page view; the controls remain usable.

Forms panel commands

- **Reset Page** restores fields on the current page to their PDF defaults.
- **Reset All** restores every field in the document.
- **Flatten Form to Copy...** creates a separate non-editable PDF with the current

answers permanently drawn onto the pages. PDF Studio verifies that no live widgets remain and never overwrites the editable original.

Form changes immediately mark the document with an unsaved *. Press **Save** or **Ctrl+S** to persist them. If you close PDF Studio or open another file while changes remain, choose **Save**, **Discard**, or **Cancel** from the warning.

Flatten only when the completed copy no longer needs to be edited. Keep the original form for later corrections.

Current limitation: multi-select list boxes are detected, but this release saves one selected item because the current PDF engine does not reliably write multiple list values.

Creating fields with Form Designer

Form Designer turns an ordinary PDF or scanned paper form into a genuine interactive AcroForm. Open the **Forms** section and enable **Design mode**. While Design mode is active, the normal fill controls are replaced by field outlines so editing the structure cannot accidentally change an answer.

1. Choose **Text Field** and drag a rectangle where typed text should appear.
2. Choose **Checkbox** and click or drag where a tick box should appear.
3. Choose **Dropdown**, place it, then use **Properties...** to enter one choice per line and decide whether users may type a custom value.
4. Choose **Date** for a DD/MM/YYYY field with an in-app calendar picker.
5. Choose **Yes / No** to create two linked radio buttons.
6. Choose **Signature** or **Initials** to create genuine unsigned PDF signature fields.
7. Choose **Select**, then drag a field to move it or drag the solid lower-right handle to resize it.

8. Use **Properties...** to set the name, tooltip, required/read-only state, and any type-specific options.

9. Use **Delete Field** to remove the selected field. Deleting either member of a Yes / No pair removes the complete linked group.

10. Press **Save**, then disable Design mode to test normal filling.

A short click creates a sensible default-sized field. Dragged and moved fields are constrained to page boundaries, and automatic names avoid collisions. Field names must remain unique.

Form Designer creates real PDF form widgets. It does not paint fake boxes on top of the page. Signature and initials controls are unsigned placeholders for signing in a compatible PDF application; PDF Studio does not yet perform certificate-backed signing.

Detecting likely fields automatically

The **Smart Form Detection** section in the Forms panel can analyse the current page before you place fields manually:

1. Choose a confidence level: **More suggestions**, **Balanced**, or **High confidence**.
2. Click **Detect Current Page...**
3. Wait for the current-page analysis to finish. PDF Studio uses the existing text layer when possible; image-only pages are read with Tesseract OCR.
4. The resizable **Review Smart Form Suggestions** window opens automatically. Each row shows the proposed type, detected label, confidence, and why it was suggested. Selecting a row highlights the matching outline on the page.
5. Untick any incorrect or unwanted suggestions. Use **Check All**, **Uncheck All**, or **Check 80%+** when useful.
6. Click **Create Checked** and confirm. The approved suggestions become genuine AcroForm fields and mark the document as unsaved.
7. If you close the review window without creating fields, reopen it with **Review Suggestions...** in the Forms panel. **Clear** removes the previews.
8. Use Form Designer to move, resize, rename, or correct the created fields.
9. Save the PDF.

Detection combines recognised labels with visible answer lines, rectangles, and checkbox squares. A suggestion supported by both text and geometry receives a higher confidence score. When PDF Studio recognises a likely label but must infer where the answer belongs, the suggestion is intentionally scored lower.

No suggestion changes the PDF until **Create Checked** is confirmed. Existing fields suppress overlapping suggestions, and **Clear** removes all previews without modifying the document. Detailed

review no longer competes for space in the navigation rail: the sidebar shows a compact status and the Forms body is scrollable when space is limited. The detector is an assistant, not an automatic claim of perfect form understanding; ambiguous layouts still require review.

11. Editing text in scanned pages

A scan is an image, not a collection of editable letters. PDF Studio therefore uses a controlled region-replacement workflow rather than claiming to provide native Word-style editing.

1. Click **Edit Text** in the **EDIT SCAN** toolbar group, or choose

Tools > Edit Scanned Text...

2. Drag a rectangle tightly around the word, number, or line to change, then release the mouse button. PDF Studio validates the visible selection and converts its display-pixel bounds into the matching PDF-page coordinates before starting OCR.

3. The status briefly reports **Selection captured**, followed by a **Preparing Scanned-Text Editor** progress window. PDF Studio uses an existing text layer when one is present. For an image-only scan, it sends only the selected region to Tesseract OCR in one recognition pass. Selected-region OCR currently uses Tesseract's English model; for other languages, type the replacement manually in the editor.

4. Review the recognised text and type the corrected replacement.

5. Adjust font size (**Auto fit** is the safest default), alignment, text colour, and background colour. The initial background is sampled from the edge pixels of the selected region.

6. Check the live replacement preview and choose an application mode.

Reversible white-out overlay

This is the recommended mode. It creates one opaque FreeText annotation over the scan while leaving the original pixels intact. The replacement can be undone with **Ctrl+Z**, redone with **Ctrl+Y**, or removed from the **Annotations** panel. It saves like an ordinary annotation.

Permanent erase and replacement

This mode removes PDF text, line art, and image pixels inside the selected rectangle, then burns in the replacement text. It cannot be undone in the current editing session. PDF Studio requires **Save As** and writes a new compact PDF under a different filename. This prevents removed content from remaining in an incremental PDF revision and preserves the original source file.

Keep selections tight. A permanent replacement removes text and blanks image pixels within the selected rectangle; overlapping vector line art may also be affected. Use extra care around nearby rules, borders, and pictures. Use the reversible overlay whenever a non-destructive correction is sufficient.

If OCR is unavailable, times out, ends unexpectedly, or cannot read the selection, the editor still opens and allows manual replacement text. Tesseract is therefore helpful but not a hard blocker for the replacement operation.

12. Redactions

1. Select the redaction tool and drag a box over the content to remove.
2. Repeat for each area.
3. **Tools > Apply Redactions.**

Applying a redaction is permanent and irreversible. The underlying text and images are removed, not merely covered. After applying, PDF Studio requires **Save As** and writes a clean file under a new name so the original remains available and removed content is not retained in an incremental revision.

13. Managing pages

From the **Pages** menu:

Action	Effect
Insert Blank Page	Adds a blank page
Delete Page	Removes the current page
Move Page Up / Down	Reorders the current page

You can also **drag and drop thumbnails** in the navigation panel to reorder pages.

All page operations are undoable (`Ctrl+Z`), including drag-reordering.

14. Merging, splitting, extracting

Tools > Merge / Split PDFs...

- **Merge** - combine several PDFs into one. Add files, set their order, choose an output path.
- **Split** - break a PDF into separate files.

Tools > Extract Pages... - pull a page range or selection into a new PDF, leaving the original untouched.

Merge and extraction write to a validated staged PDF before replacing the chosen destination. Split stages and validates every part before committing the set. If a later part cannot be committed, earlier replacements are rolled back. Closing a working dialog requests cancellation and waits for the worker to stop instead of destroying a live thread.

15. Password protection

Tools > Password Protect...

- Set an **open password** (required to view the document).
- Set a **permissions password** (controls what may be done with it).
- Toggle permissions: printing, copying, annotating, form filling, and more.
- Encryption is **AES-256**.

Protection is applied when you save.

If you lose the password, the document cannot be recovered. There is no back door.

16. OCR - making scans searchable

A scanned page is just an image: you cannot search or copy from it. OCR adds an **invisible text layer** beneath the image, so the page looks identical but becomes searchable and selectable.

Tools > Run OCR...

1. Choose the scope: **all pages**, **current page**, or a **custom range**.
2. Choose the **language** (from your installed Tesseract language packs).
3. Choose to **save as a new file** or **overwrite the original**.
4. Run. Processing happens in the background with a progress bar - the app

stays usable.

Requirements: `pytesseract` and `Pillow` are bundled with PDF Studio. Install **Tesseract-OCR** normally; PDF Studio detects and configures it directly, so no system `PATH` editing is required. If automatic detection misses a custom installation, use **Locate Tesseract...** in the OCR dialog. Poppler is no longer required because PDF Studio renders pages directly through PyMuPDF.

17. Exporting pages and document content

Choose **File > Export As**.

Image files

Image Files (.png, .jpg, .webp, .tiff, .bmp, .gif)... renders the visible PDF page exactly as an image. This is ideal for posters, flyers, social-media artwork, previews, and attaching a PDF page where an image is required.

1. Choose **All pages**, **Current page**, or enter a range such as `1-3, 6, 9-10` .
2. Choose PNG, JPEG, WebP, TIFF, BMP, or GIF.
3. Choose the resolution. **300 DPI** is a strong default for posters and printing; **150 DPI** is usually enough for screen use.
4. For JPEG and WebP, choose the quality. For PNG, WebP, and TIFF, you may keep a transparent page background where the PDF permits it.
5. Click **Export**.

A single page is saved to one image file. Multiple pages are written as numbered files such as `poster_page_001.png` . GIF export is static: multiple pages become separate GIF files rather than an animation.

PDF Studio stages all selected pages before replacing destination files. If a page fails or the run is cancelled, it does not leave a half-created export set. Word and Excel exports now follow the same principle: the generated DOCX/XLSX is validated as an OOXML archive before it can replace an existing destination.

Microsoft Word

Microsoft Word (.docx) preserves layout, text, images, and columns via `pdf2docx` . You can select a continuous page range.

Microsoft Excel

Microsoft Excel (.xlsx) extracts *tables* into styled worksheets (headers, alternating row shading, frozen panes) via `tabula-py` . Requires **Java**.

All export types run in the background with a progress bar.

Word/Excel quality depends on the source. A clean, text-based PDF converts well; a scanned or heavily designed one may need cleanup. Excel export finds tables - it is not a general PDF-to-spreadsheet converter.

18. Saving and printing

Action	Shortcut	Notes
Save	Ctrl+S	Writes changes to the current file
Save As...	Ctrl+Shift+S	Saves to a new file
Save a Copy...	-	Writes a copy, keeps editing the original
Print Preview...	Ctrl+Shift+P	See exactly what will print, before printing
Print...	Ctrl+P	Any system printer
Properties	-	View document metadata

Print Preview (Ctrl+Shift+P) shows exactly what will come out of the printer before anything is sent, so you can check the pages and orientation without wasting paper. The preview is produced by the same rendering code as the actual print, so what you see is what you get.

Annotations, markup, signatures, stamps, and form entries are all embedded on save. Same-file save preparation is snapshot-protected: if preparation or the incremental write fails, PDF Studio restores the pre-save in-memory document. An asterisk (*) in the title bar indicates unsaved changes.

The PDF and its application sidecars commit as one bundle. Original transaction copies are kept under PDF Studio's controlled local application-data recovery folder, not beside the document. If a commit fails and every destination is restored, PDF Studio reports a normal save failure and keeps the document unsaved. If automatic restoration is incomplete, **Save Rollback Incomplete** shows the recovery path and preserves the original copies for inspection.

After a successful save, PDF Studio verifies that the recovery transaction was removed. If antivirus, synchronisation, preview, or backup software leaves any original copy locked, **Recovery Copies Still Present** appears with **Retry Deletion** and **Open Recovery Folder** actions. Those copies may contain sensitive pre-redaction content, so PDF Studio does not describe a secure redaction as complete until cleanup is confirmed.

19. File associations (Windows)

To make Windows open PDFs in PDF Studio:

File > Set as Default PDF App...

This registers PDF Studio (per-user; **no administrator rights required**) and opens the Windows *Default apps* page, where you select **PDF Studio** for .pdf .

Alternatively: right-click a PDF > **Open with** > **Choose another app** > **PDF Studio** > tick *Always use this app*.

Windows 10/11 deliberately prevent applications from silently making themselves the default handler (anti-hijacking). A one-time manual confirmation is therefore always required - by design, not a limitation of this app.

Command line:

```
"PDF Studio.exe" --register           :: .pdf + Word/Excel
"PDF Studio.exe" --register --pdf-only :: .pdf only
"PDF Studio.exe" --unregister         :: remove associations
```

Or use `register_pdf.bat` / `unregister_pdf.bat` .

20. Keyboard shortcuts

File

Shortcut	Action
Ctrl+O	Open
Ctrl+S	Save
Ctrl+Shift+S	Save As
Ctrl+P	Print
Ctrl+Shift+P	Print preview
Ctrl+Q	Quit

Edit

Shortcut	Action
Ctrl+Z	Undo
Ctrl+Y / Ctrl+Shift+Z	Redo
Ctrl+C	Copy selected text
Ctrl+F	Find
F3 / Shift+F3	Find next / previous

Navigation

Shortcut	Action
< / >	Previous / next page
Ctrl+< / Ctrl+>	Previous / next page
Ctrl+Home / Ctrl+End	First / last page
Ctrl+B	Add bookmark

View

Shortcut	Action
Ctrl++ / Ctrl+-	Zoom in / out
Ctrl+Wheel	Zoom in / out
Ctrl+Shift+H	Fit width
Ctrl+Shift+F	Fit page
Ctrl+R	Rotate 90°
F11	Full screen
F4	Toggle navigation panel
Esc	Cancel the active tool

21. Building from source

Build in a **clean, isolated Python 3.11 environment**. The supported internal build path is:

```
build_clean.bat
```

It creates a fresh `.buildenv`, installs the declared build dependencies, runs `pip check`, executes the full test suite, captures exact package versions, runs the release audit, generates the build manifest, and invokes PyInstaller. Any failed command stops the build. The internal executable is left in `src\dist\`.

For day-to-day work, run `setup.bat` once, then use `buildit.bat`. It performs the same validation gates using `.venv`.

Release-assurance utilities:

- `capture_release_environment.bat` records exact versions from a passing Windows environment.
- `prepare_release_wheelhouse.bat` downloads those versions and records SHA-256 hashes.
- `release_check.bat` checks the public-release policy and deliberately fails while licensing approval is unresolved.
- `build_release.bat` uses the verified offline wheelhouse and is available only after every public-release gate passes.

Always use `python -m PyInstaller` , **never a bare** `pyinstaller` . The bare command may use a global installation or the wrong environment.

Application metadata is centralised in `src/app_metadata.py` . The executable name and bundled release resources are defined in `src/PDF Studio.spec` .

22. Troubleshooting

Diagnostics and logs

Choose **Help > Diagnostics...** to copy a support report or open the rotating log folder. The report includes application, runtime, dependency, Tesseract, and build information, but no PDF document text. Review file paths before sharing it.

Choose **Help > Third-Party Licences and Notices...** to inspect the bundled dependency inventory and current distribution warning.

A save reports “Save Rollback Incomplete” Stop editing the affected PDF and sidecar files. The dialog gives the recovery directory containing original `.original` copies and `recovery_manifest.json` . Keep that folder, open **Help > Diagnostics...**, and inspect the log before manually restoring or replacing any affected file. PDF Studio deliberately does not claim that destinations were preserved when automatic rollback could not finish.

A Word/Excel document won't open Install **Microsoft Office** (best fidelity) or **LibreOffice** (free). Large documents take a few seconds to convert.

"Missing pdf2docx" when exporting to Word (built .exe) `pdf2docx` wasn't installed in the environment the executable was built from. Install it and rebuild. The error dialog now reports the underlying import failure, which names the specific missing module.

"bootstrap.ini is corrupt" when opening a Word file from the .exe Fixed in v2.8. External programs are now launched with a cleaned environment. If you see this, you are running an older build - rebuild from current source.

Scanned PDF can't be searched It has no text layer. Run **Tools > Run OCR...** (§16).

The scan text replacement does not match the original font PDF Studio estimates a safe Helvetica size and background colour, but a scan does not contain reusable font information. Adjust

font size, alignment, and colours in the preview. Use the reversible overlay first when matching is uncertain.

OCR fails Open **Tools > Run OCR...** and check the OCR engine panel. PDF Studio searches standard Tesseract install locations automatically and does not require a `PATH` change. Use **Locate Tesseract...** for a custom install, or **Get Tesseract** if it is not installed. Poppler is not required.

Excel export fails Confirm **Java** is installed.

Build aborts: "multiple Qt bindings packages" PyQt5 is visible in your build environment. Build in a clean venv (`build_clean.bat`).

Text is too small / hard to read **View > Appearance > Text Size > Extra Large**, and try both themes (§4).

The mouse behaves oddly on the page A markup tool is active. Press `Esc` .

23. Licence and credits

PDF Studio - Copyright © 2025-2026 Leon Priest. The application source currently declares the **Apache License, Version 2.0**. See `LICENSE.txt` .

Third-party components

Component	Licence choices / status
PyMuPDF / MuPDF	AGPL-3.0 or commercial terms (Artifex)
PyQt6	GPL-3.0 or Riverbank commercial terms
Qt 6	Separate Qt licensing applies
Pillow	Pillow/HPND-style licence
pytesseract	Apache-2.0
Atkinson Hyperlegible	SIL Open Font License 1.1 - © 2020 Braille Institute of America

Optional packages may include `pdf2docx` , `tabula-py` , `openpyxl` , `pandas` , and `pywin32` .

See `NOTICE` , `THIRD_PARTY_NOTICES.md` , and the bundled `licenses` folder.

Distribution gate: a bundled binary is not approved for family or public distribution until the project owner deliberately resolves the PyMuPDF/MuPDF, PyQt6, and Qt licensing strategy and records approval in `release/release_policy.json` . `release_check.bat` enforces this gate. This is an engineering safeguard, not legal advice.